

INF-2300: COMPUTER COMMUNICATION

PROJECT 3

Eivind Østlyngen

eos033@uit.no

November 11, 2022

1 Introduction

In this report, I will describe and discuss my implementation of the go-back-n algorithm used to perform reliable data transport in the transport layer. I will also compare this algorithm to other alternatives.

2 Technical background

2.1 Transport layer

The transport layer is a layer in the Internet Protocol(IP) stack with the responsibility of assuring communication and data delivery between two applications on different host machines. [1]

3 Design & Implementation

For this assignment we were given a simulation to simulate packet transfers from one host to another. Packets from both hosts, the sender and the receiver, had the possibility of being delayed, corrupted or lost making it unreliable data transfer. With the goal of making the transfer reliable, the go-back-n algorithm was implemented. The go-back-n is a sliding window protocol, meaning it tracks all outgoing packets until it has been confirmed received by the other host. We track this window by using two pointers, the base telling us where it starts and $\text{base} + N$ which is the end. Where N is the size of the window. We also have a third pointer, nextseqnum , which points to the first packet that has not been sent yet but is contained in the window. Each of the packets are marked with a sequence number which states the order they are sent out in.

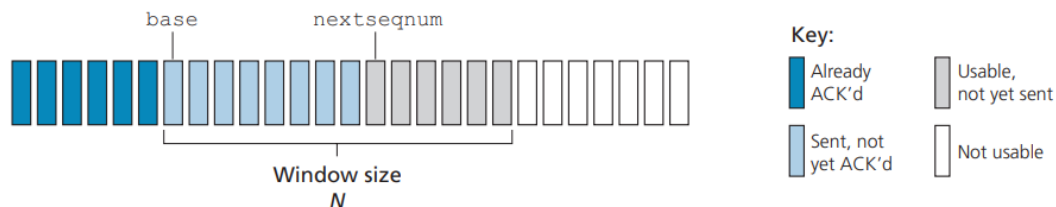


Figure 1: [1]Showcasing the segmentation of the different stages of packets in the window

When the sender first receives binary data from the application layer, it creates a packet containing the data and marks it with the first available sequence number, nextseqnum . nextseqnum is then incremented for the next packet. The packet is added to the window, then starts a timer and sends it to the other host. The receiver. If the host receives the packet before the timer runs out, meaning it was not delayed or lost, we check if the binary data inside is corrupted. From the assignment text we are told that the expected data is only

capital alphabetic letters. Therefore we can check if a packet is corrupted by simply extracting the data, and performing a regex check against these conditions. If the packet is not corrupted, we receive the packet by sending it on to the application layer and the transfer was a success. As the packet was received, we also reset the timer to track if the next sent packet arrives correctly. After passing the data, an acknowledge packet is sent back to the sender, indicating that the packet was received successfully. If the sender receives the acknowledgement packet, which also can be lost or delayed, we increment the base pointer and therefore shifting the window.

If the packet is lost or delayed, the packet is not received before the timer runs out, a timeout function is called. This timeout function resets the timer and then sends all packets that has previously been sent but not acknowledged. The reason we send all packets, even though only one has been lost is because we want to receive the packets in the correct order. As the go-back-n algorithm allows the sender to send multiple packets without receiving an acknowledgement, there is a possibility that the receiver receives packets in the wrong order. To prevent wrong order, we simply ignore packets that do not contain the expected sequence number. As the packet is ignored the base is not shifted, so the missing packet will be sent again in the timeout function.

As the acknowledgement packages can also be lost or delayed, the algorithm is implemented using cumulative acknowledgement. This means that if multiple packets are sent from the sender, and is received properly, but only the last acknowledgement packet is received correctly from the sender, the window still increments as all packets were received. The sender can assume this because the receiver tracks the expected sequence numbers, and would not send an acknowledgement if a packet is out of order.

4 Discussion

4.1 Testing

The implementation has been tested several times using the simulator, with different values, and passing. The highest tested values are 0.5 on corrupt, drop and delay chance using 25 packets and a random seed. This means that there is a 0.5^3 (12.5%) chance that a packet is sent successfully, which is way above values one can expect in a realistic scenario and the implementation still works as expected.

One issue that could arise is if a packet gets corrupt but does not get caught by the implementation. As discussed earlier the current implementation checks for all capital a-z letters, but there is no documentation in the assignment text on how the packets are corrupted. Therefore there could exist a case where a packet is corrupted and still only contain capital a-z letters. This issue has not been encountered and therefore assumed to not be a possible case.

4.2 Go-back-n vs selective repeat

Another algorithm designed to solve the same issue is selective repeat. Similarly to go-back-n the packets are buffered in a window on the senders side, but using selective repeat also requires buffering packets on the receivers side. The packets that are buffered are the out of order packets. If one of N packets is lost, instead of sending the entire window again only the lost packet is sent again. When the lost packet arrives, we take the out of order packets from the buffer and send them all in the correct order to the application layer at the same time. As we do not send the entire window when a packet is lost/corrupt, we end up sending a lot less packets, especially if there is a high chance a packet is corrupted/lost. As this algorithm does not have cumulative acknowledgements, the sender also needs to track which packets in the window are acknowledged.

In conclusion, for this scenario selective repeat would be a significant performance boost due to the high number of retransmissions of go-back-n. This performance increase comes at a cost of a higher complexity implementation and more memory required to buffer on both hosts.

4.3 Rejecting data

According to the text book for this course, when we receive binary data and the window is full we are supposed to reject the data and the binary data would be attempted to be sent again later. This is not supported in this implementation as the simulation only sends the binary data to the transport layer once before fetching the next chunk and discarding the last. We were also told to specifically only adjust the implementation of the transport layer in the assignment text. Therefore to prevent losing the binary data if the window is full, we store them in the same array as the window to be accessed later when the window slides. These packets are not accessible before the window shifts to their value.

5 Conclusion

In conclusion the implementation meets all the specified requirements and works as expected.

6 Sources

References

- [1] Kurose, J. and Ross, K. (2021) Computer networking: A top-down approach. Boston, Massachusetts: Pearson.